

PYTHON

Module 26: Modular Programming & Libraries

CODING

Introduction to Python Modules

A module is essentially a code library—a file containing a set of functions and variables that you want to include in your application. At CodeHive, we use modules to maintain clean, scalable, and organized codebases.

- ****The .py Extension****: Any Python file is technically a module. Simply save your logic in a file like `utility.py` to make it a module.

Importing & Usage Syntax

To use a module's content in another script, use the `import` keyword.

- ****Standard Access****: Use the dot notation (`module.function()`) to access elements. This prevents naming conflicts with your local script.
-

- ****Aliasing****: Use the `as` keyword to create a shorter nickname for a module. Example: `import pandas as pd`.
-

Selective Imports

If you only need a specific function or dictionary from a large module, use the `from` keyword.

• **Syntax**: `from mymodule import person1` allows you to use `person1` directly without the `mymodule.` prefix.

• **Pro-Tip**: Selective imports save memory and make your code more explicit about what dependencies it actually uses.

Built-in & Introspection Tools

Python comes with a 'Batteries Included' philosophy, providing hundreds of built-in modules like `math`, `platform`, and `random`.

- **The `dir()` function**: A powerful introspection tool that lists every function, variable, and class defined within a module. This is essential for exploring newly imported libraries.

Module Architecture at CodeHive

• ****Variable Storage****: Modules aren't just for functions; they are excellent places to store shared dictionaries, configuration constants, and global objects.

• ****Hierarchy****: As your project grows, modules can be grouped into 'Packages' (folders with an `__init__.py` file) for even deeper organization.
